

3.8. Oracle jet form with Multi-Step Form with Validation, oj-train, oj-avatar, and File Upload

Introduction:

In this hands-on activity, you will create a complete **multi-step user profile form** using Oracle JET components. The form will capture personal information, user preferences, and allow for a final review and submission. Key JET components such as <oj-train>, <oj-avatar>, <oj-c-file-picker>, <oj-form-layout>, and form validation mechanisms will be used to guide users smoothly through the process.

This form simulates a real-world data collection workflow and demonstrates how to build a **wizard-style interface** using MVVM architecture with Knockout.js and Oracle JET.

Learning Objectives:

By the end of this activity, you will be able to:

1. Create and manage **multi-step form navigation** using <oj-train> and observable step tracking.
2. Implement **data binding** for various Oracle JET form components (e.g., oj-input-text, oj-select-single, oj-radioset, oj-checkboxset).
3. Dynamically populate dropdowns using **dependent data providers** (e.g., country → state).
4. Use <oj-c-file-picker> for **file upload functionality** with custom event handling.
5. Show **user initials** dynamically with <oj-avatar>, computed from the entered name.
6. Apply **validation** on form submission and use <oj-messages> to show error/success messages.
7. Use <oj-dialog> to show a submission confirmation popup.
8. Display the submitted data as JSON using Knockout's data binding.
9. Add **export functionality** to download the submitted data as a .json file.
10. Reset form data and reinitialize form state programmatically.

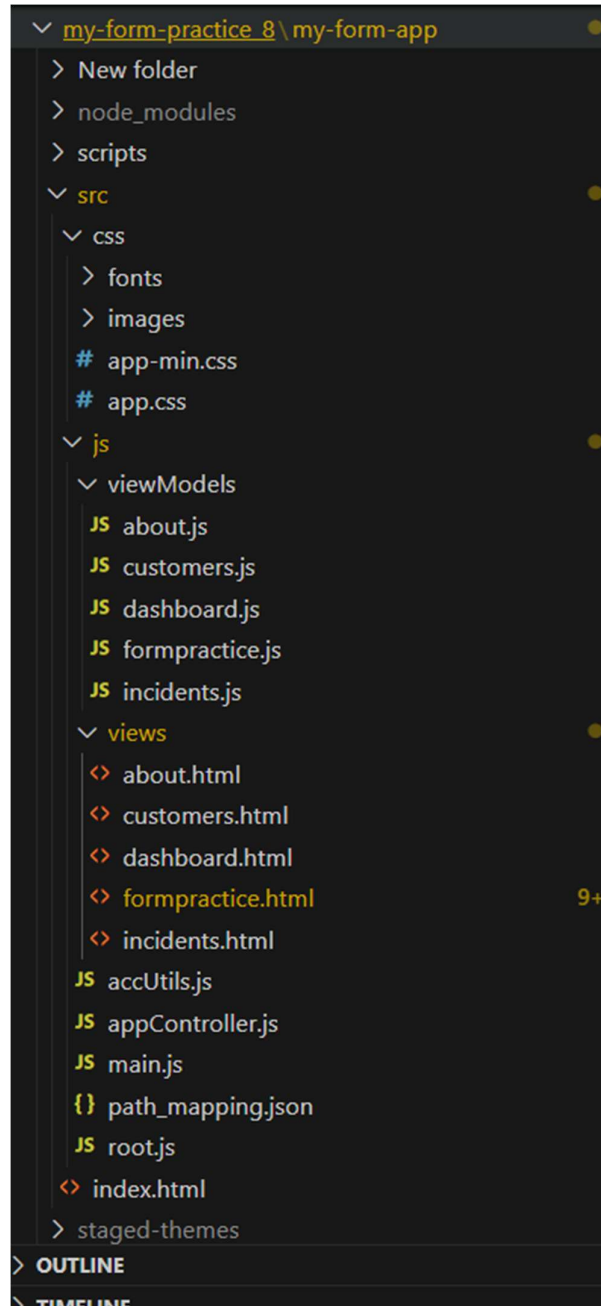
Pre-requisites:

To successfully complete this activity, you should already be comfortable with:

- Oracle JET application structure using the **NavDrawer template**.
- Basic understanding of **MVVM architecture** with Knockout observables and bindings.
- Using **Oracle JET form components** like <oj-input-text>, <oj-select-single>, and <oj-form-layout>.

- JavaScript array manipulation and event handling.
- Creating and binding **custom functions** to handle navigation, submission, and state management.

Project structure



1. Create a new folder named "my-form-app-practice_8" and move to that directory
2. In the command line run,

Ojet create my-form-app --template=navdrawer

3. Then run ojet build web and ojet serve respectively (like we did before)

4. Create folders /js/viewModels/formpractice.js and /js/views/formpractice.html

5. Add code to formpractice.js

```
my-form-practice_8 > my-form-app > src > js > viewModels > JS formpractice.js > define() callback > FormPracticeView
1  define([
2    'knockout',
3    'ojs/ojarraydataprovider',
4    'ojs/ojformlayout',
5    'ojs/ojinputtext',
6    'ojs/ojselectsingle',
7    'ojs/ojradioset',
8    'ojs/ojcheckboxset',
9    'ojs/ojbutton',
10   'ojs/ojdialog',
11   'ojs/ojmessages',
12   'ojs/ojtrain',
13   'ojs/ojavatar',
14   'ojs/ojlabel',
15   'oj-c/file-picker',
16   'ojs/ojtoolbar',
17   'ojs/ojprogressbar',
18   'ojs/ojlabelvalue'
19 ], function (ko, ArrayDataProvider) {
20   function FormPracticeViewModel() {
21     this.trainSteps = [
22       { id: 'step1', label: 'Personal Information' },
23       { id: 'step2', label: 'User Preferences' },
24       { id: 'step3', label: 'Review and Submit' }
25     ];
26     this.stepValue = ko.observable('step1');
27
28     // Step 1: Personal Info Observables
29     this.name = ko.observable('');
30     this.email = ko.observable('');
31     this.selectedCountry = ko.observable('');
32     this.selectedState = ko.observable('');
33     this.step1Messages = ko.observableArray([]); // For validation messages in Step 1
34
35     // Step 2: Preferences Observables
36     this.gender = ko.observable('');
37     this.hobbies = ko.observableArray([]);
```

```

38     this.profilePhoto = ko.observableArray([]);
39
40     // Properties for oj-c-file-picker
41     this.acceptArr = ko.observableArray([]);
42     this.multipleStr = ko.observable('single');
43     this.isDisabled = ko.observable(false);
44
45     // Data for Country and State dropdowns
46     this.countryOptions = [
47         { value: 'IN', label: 'India' },
48         { value: 'US', label: 'USA' },
49         { value: 'UK', label: 'UK' }
50     ];
51     this.countryDataProvider = new ArrayDataProvider(this.countryOptions, { keyAttributes: 'value' });
52
53     this.stateMap = {
54         IN: [
55             { value: 'PB', label: 'Punjab' },
56             { value: 'DL', label: 'Delhi' },
57             { value: 'MH', label: 'Maharashtra' }
58         ],
59         US: [
60             { value: 'NY', label: 'New York' },
61             { value: 'CA', label: 'California' },
62             { value: 'TX', label: 'Texas' }
63         ],
64         UK: [
65             { value: 'ENG', label: 'England' },
66             { value: 'SCT', label: 'Scotland' },
67             { value: 'WLS', label: 'Wales' }
68         ]
69     };
70
71     this.stateDataProvider = ko.pureComputed(() => {
72         const country = this.selectedCountry();
73         const states = this.stateMap[country] || [];
74         return new ArrayDataProvider(states, { keyAttributes: 'value' });
75     });
76
77     this.isStateEnabled = ko.computed(() => !!this.selectedCountry());
78
79     // Avatar Initials (derived from name)
80     this.avatarInitials = ko.computed(() => {
81         const name = this.name();
82         if (!name) return '';
83         const parts = name.trim().split(' ');
84         if (parts.length === 1) return parts[0].charAt(0).toUpperCase();
85         if (parts.length >= 2) return (parts[0].charAt(0) + parts[parts.length - 1].charAt(0)).toUpperCase();
86         return '';
87     });
88
89     // Form Submission Status and Data
90     this.formSubmitted = ko.observable(false);
91     this.submittedData = ko.observable({});
92     this.submissionProgress = ko.observable(0); // Value for progress bar
93     this.isSubmitting = ko.observable(false); // Controls progress bar visibility
94     this.submissionMessages = ko.observableArray([]); // For messages in review step
95
96     // Navigation Functions
97     this.goToNextStep = () => {
98         if (this.stepValue() === 'step1') {
99             const form1 = document.getElementById('personalInfoForm');
100             // Validate Step 1 form before proceeding
101             if (form1 && !form1.checkValidity()) {
102                 this.step1Messages.push({
103                     severity: 'error',
104                     summary: 'Validation Error',

```

```

105         detail: 'Please fill in all required fields in Personal Information.'
106     });
107     form1.reportValidity(); // Shows native browser validation messages
108     return; // Stop if form is invalid
109 }
110 this.step1Messages([]); // Clear any previous validation messages
111 this.stepValue('step2'); // Move to Step 2
112 } else if (this.stepValue() === 'step2') {
113     // You can add validation for Step 2 here if needed
114     this.stepValue('step3'); // Move to Step 3
115 }
116 };
117
118 this.goToPreviousStep = () => {
119     if (this.stepValue() === 'step2') {
120         this.stepValue('step1'); // Move to Step 1
121     } else if (this.stepValue() === 'step3') {
122         this.stepValue('step2'); // Move to Step 2
123     }
124 };
125
126 // File Picker Event Handlers
127 this.selectListener = (event) => {
128     console.log("Files selected:", event.detail.files);
129     this.profilePhoto(event.detail.files); // Store selected files
130 };
131
132 this.invalidListener = (event) => {
133     console.log("Invalid file selected:", event.detail);
134     // You can add a message to submissionMessages here if needed
135     this.submissionMessages.push({
136         severity: 'warning',
137         summary: 'Invalid File',
138         detail: 'The selected file type is not allowed or too large.'
139     });
140 };
141
142 // Form Submission Handler
143 this.handleSubmit = (event) => {
144     if (event) {
145         event.preventDefault();
146     }
147
148     if (this.stepValue() === 'step3') {
149         this.isSubmitting(true); // Show progress bar
150         this.submissionProgress(null); // Set to indeterminate progress
151
152         // Simulate an asynchronous submission (e.g., API call)
153         setTimeout(() => {
154             const formData = {
155                 name: this.name(),
156                 email: this.email(),
157                 country: this.selectedCountry(),
158                 state: this.selectedState(),
159                 gender: this.gender(),
160                 hobbies: this.hobbies(),
161                 profilePhotoName: this.profilePhoto().length > 0 ? this.profilePhoto()[0].name : 'None selected'
162             };
163
164             console.log("Form Submitted:", formData); // Log submitted data
165             this.submittedData(formData); // Store submitted data for review
166             this.formSubmitted(true); // Indicate form was submitted
167             this.submissionProgress(0); // Reset progress bar value
168             this.isSubmitting(false); // Hide progress bar

```

```

169
170 // Display success message
171 this.submissionMessages.push({
172   severity: 'confirmation',
173   summary: 'Submission Successful',
174   detail: 'Your form has been submitted.'
175 });
176
177 // Open the success dialog
178 const dialog = document.getElementById('submitDialog');
179 if (dialog) {
180   dialog.open();
181 }
182 }, 1500); // Simulate 1.5 seconds delay for submission
183 }
184 };
185
186 // Reset Form Functionality
187 this.resetForm = () => {
188   this.name('');
189   this.email('');
190   this.selectedCountry('');
191   this.selectedState('');
192   this.gender('');
193   this.hobbies([]);
194   this.profilePhoto([]);
195   this.formSubmitted(false);
196   this.submittedData({});
197   this.submissionMessages([]); // Clear all messages
198   this.step1Messages([]); // Clear step 1 messages
199   this.submissionProgress(0);
200   this.isSubmitting(false);

```

```

200     this.isSubmitting(false);
201     this.stepValue('step1'); // Reset train to first step
202 };
203
204 // Close Submission Dialog
205 this.closeDialog = () => {
206     const dialog = document.getElementById('submitDialog');
207     if (dialog) {
208         dialog.close();
209     }
210 };
211
212 // Handle direct train step clicks
213 this.handleTrainStepChange = (event) => {
214     this.stepValue(event.detail.value);
215 };
216
217 // Export Data Functionality
218 this.exportData = () => {
219     const data = {
220         name: this.name(),
221         email: this.email(),
222         country: this.selectedCountry(),
223         state: this.selectedState(),
224         gender: this.gender(),
225         hobbies: this.hobbies(),
226         profilePhotoName: this.profilePhoto().length > 0 ? this.profilePhoto()[0].name : 'None'
227     };
228     const jsonData = JSON.stringify(data, null, 2); // Pretty print JSON
229     const blob = new Blob([jsonData], { type: 'application/json' });
230     const url = URL.createObjectURL(blob); // Create a URL for the blob
231     const a = document.createElement('a'); // Create a temporary anchor element
232     a.href = url;
233
234     a.download = 'formData.json'; // Set the download file name
235     document.body.appendChild(a); // Append to body (needed for Firefox)
236     a.click(); // Programmatically click the link to trigger download
237     document.body.removeChild(a); // Clean up the temporary link
238     URL.revokeObjectURL(url); // Release the object URL
239     console.log('Form data exported as formData.json');
240 };
241
242 return new FormPracticeViewModel();
243 });

```

Richa J. Arora (1 day ago) Ln 72, Col 46 Space

6. Now add formpractice.html

formpractice.html 9+ X

my-form-practice_8 > my-form-app > src > js > views > formpractice.html > div.oj-hybrid-padding > div.s

```
1 <div class="oj-hybrid-padding">
2   <style>
3     /* Styles for positioning the avatar */
4     .avatar-container {
5       position: absolute;
6       top: 130px; /* Adjust as needed for vertical position */
7       right: 100px; /* Adjust as needed for horizontal position */
8       z-index: 10; /* Ensure it's above other content if overlapping */
9     }
10    /* Basic spacing for step content */
11    .step-content {
12      margin-top: 20px;
13    }
14    /* Margin for review items */
15    .review-item {
16      margin-bottom: 10px;
17    }
18    /* Style for progress bar within toolbar */
19    .my-progress-in-toolbar {
20      flex-grow: 1;
21      margin-left: 15px;
22      max-width: 200px;
23    }
24  </style>
25
26  <div class="oj-flex">
27    <oj-train id="profileTrain"
28      steps="[[trainSteps]]"
29      selected-step="{{stepValue}}"
30      on-selected-step-changed="[[handleTrainStepChange]]">
31      <oj-train-step id="step1" label="Personal Information"></oj-train-step>
32      <oj-train-step id="step2" label="User Preferences"></oj-train-step>
33      <oj-train-step id="step3" label="Review and Submit"></oj-train-step>
34    </oj-train>
35  </div>
36
37  <div class="step-content" data-bind="if: stepValue() === 'step1'">
```

```

38 <h2>Enter Your Personal Details</h2>
39 <oj-messages messages="{{step1Messages}}"></oj-messages>
40 <form id="personalInfoForm" class="oj-form-layout" max-columns="2" direction="row" onsubmit="return false;">
41   <oj-label for="nameInput">Enter name</oj-label>
42   <oj-input-text id="nameInput" label-hint="Enter name" required="true" value="{{name}}"></oj-input-text>
43
44   <oj-label for="emailInput">Enter email</oj-label>
45   <oj-input-text id="emailInput" label-hint="Enter email" required="true" value="{{email}}"></oj-input-text>
46
47   <oj-label for="countrySelect">Select Country</oj-label>
48   <oj-select-single id="countrySelect" label-hint="Select Country"
49     data="[[countryDataProvider]]"
50     value="{{selectedCountry}}"
51     required="true">
52   </oj-select-single>
53
54   <oj-label for="stateSelect">Select State</oj-label>
55   <oj-select-single id="stateSelect" label-hint="Select State"
56     data="[[stateDataProvider]]"
57     value="{{selectedState}}"
58     required="true"
59     disabled="[[!isStateEnabled()]]">
60   </oj-select-single>
61 </form>
62 </div>
63
64 <div class="step-content" data-bind="if: stepValue() === 'step2'">
65   <h2>Tell Us About Your Preferences</h2>
66   <oj-form-layout>
67     <div>
68       <oj-label for="genderGroup">Gender</oj-label>
69       <oj-radioset id="genderGroup" value="{{gender}}" required="true">
70         <oj-option value="male">Male</oj-option>
71         <oj-option value="female">Female</oj-option>
72         <oj-option value="other">Other</oj-option>
73       </oj-radioset>
74     </div>
75
76     <div>
77       <oj-label for="hobbiesGroup">Hobbies</oj-label>
78       <oj-checkboxset id="hobbiesGroup" value="{{hobbies}}">
79         <oj-option value="reading">Reading</oj-option>
80         <oj-option value="sports">Sports</oj-option>
81         <oj-option value="music">Music</oj-option>
82       </oj-checkboxset>
83     </div>
84
85     <div>
86       <oj-label for="profilePhoto">Profile Photo</oj-label>
87       <oj-c-file-picker id="profilePhoto"
88         accept="[[acceptArr]]"
89         selection-mode="[[multipleStr]]"
90         on-oj-select="[[selectListener]]"
91         on-oj-invalid-select="[[invalidListener]]"
92         disabled="[[isDisabled]]"
93         label="Upload"></oj-c-file-picker>
94       <span data-bind="text: profilePhoto().length > 0 ? profilePhoto()[0].name : ''"></span>
95     </div>
96   </oj-form-layout>
97 </div>
98
99 <div class="step-content" data-bind="if: stepValue() === 'step3'">
100   <h2>Review Your Information</h2>
101   <oj-messages messages="{{submissionMessages}}"></oj-messages>
102   <div class="oj-form-layout">
103     <oj-label-value label-edge="start" label-width="30%" class="review-item">

```

```

104 <oj-label slot="label" for="reviewName">Name</oj-label>
105 <oj-input-text slot="value" id="reviewName" value="{{name}}" readonly="true"></oj-input-text>
106 </oj-label-value>
107 <oj-label-value label-edge="start" label-width="30%" class="review-item">
108 <oj-label slot="label" for="reviewEmail">Email</oj-label>
109 <oj-input-text slot="value" id="reviewEmail" value="{{email}}" readonly="true"></oj-input-text>
110 </oj-label-value>
111 <oj-label-value label-edge="start" label-width="30%" class="review-item">
112 <oj-label slot="label" for="reviewCountry">Country</oj-label>
113 <oj-input-text slot="value" id="reviewCountry" value="{{selectedCountry}}" readonly="true"></oj-input-text>
114 </oj-label-value>
115 <oj-label-value label-edge="start" label-width="30%" class="review-item">
116 <oj-label slot="label" for="reviewState">State</oj-label>
117 <oj-input-text slot="value" id="reviewState" value="{{selectedState}}" readonly="true"></oj-input-text>
118 </oj-label-value>
119 <oj-label-value label-edge="start" label-width="30%" class="review-item">
120 <oj-label slot="label" for="reviewGender">Gender</oj-label>
121 <oj-input-text slot="value" id="reviewGender" value="{{gender}}" readonly="true"></oj-input-text>
122 </oj-label-value>
123 <oj-label-value label-edge="start" label-width="30%" class="review-item">
124 <oj-label slot="label" for="reviewHobbies">Hobbies</oj-label>
125 <oj-input-text slot="value" id="reviewHobbies" value="{{hobbies().join(', ')}}" readonly="true"></oj-input-text>
126 </oj-label-value>
127 <oj-label-value label-edge="start" label-width="30%" class="review-item">
128 <oj-label slot="label" for="reviewPhoto">Profile Photo</oj-label>
129 <oj-input-text slot="value" id="reviewPhoto" value="{{profilePhoto().length > 0 ? profilePhoto()[0].name : 'None selected'}}" readonly="true"></oj-input-text>
130 </oj-label-value>
131 </div>
132
133 <oj-toolbar>
134 <oj-button on-oj-action="[[handleSubmit]]">
135 <span slot="startIcon" class="oj-ux-ico-check"></span>
136
137 </oj-button>
138 <oj-button on-oj-action="[[resetForm]]">
139 <span slot="startIcon" class="oj-ux-ico-undo"></span>
140 Reset
141 </oj-button>
142 <oj-button on-oj-action="[[exportData]]">
143 <span slot="startIcon" class="oj-ux-ico-export"></span>
144 Export
145 </oj-button>
146
147 </oj-toolbar>
148 </div>
149
150 <div class="avatar-container">
151 <oj-avatar id="userAvatar"
152 role="img"
153 aria-label="User avatar"
154 size="lg"
155 initials="[[avatarInitials]]">
156 </oj-avatar>
157 </div>
158
159 <div class="oj-flex oj-sm-justify-content-center oj-sm-margin-top-4" data-bind="if: stepValue() !== 'step3'">
160 <oj-button data-bind="click: goToPreviousStep, visible: stepValue() !== 'step1'">Back</oj-button>
161 <oj-button data-bind="click: goToNextStep, visible: stepValue() !== 'step3'" class="oj-sm-margin-start-2">Next</oj-button>
162 </div>
163
164 <oj-dialog id="submitDialog" dialog-title="Submission Successful" modality="modeless">
165 <div slot="body">
166 <p>Your form was submitted successfully!</p>
167 </div>
168 <div slot="footer">
169 <oj-button on-oj-action="[[closeDialog]]">OK</oj-button>
170 </div>
171 </oj-dialog>
172
173 <div class="oj-sm-margin-top-4" data-bind="if: formSubmitted">
174 <h3>Submitted Data (JSON)</h3>
175 <pre data-bind="text: JSON.stringify(submittedData(), null, 2)"></pre>
176 </div>
177 </div>

```

Here we have added the toolbar that contains the submit, reset and back buttons and also, after we close the submit dialog box , the green colour confirmation box will appear..

1

2

3

Personal InformationUser PreferencesReview and Submit

RJ

Review Your Information

✓ Confirmation

✕

Submission Successful
Your form has been submitted.

Name	richa jain
Email	demo@gmail.com
Country	IN
State	MH
Gender	female
Hobbies	reading
Profile Photo	None selected

✓ Submit

↶ Reset

📄 Export

Submitted Data (JSON)

```
{  "name": "richa jain",  "email": "demo@gmail.com"}
```